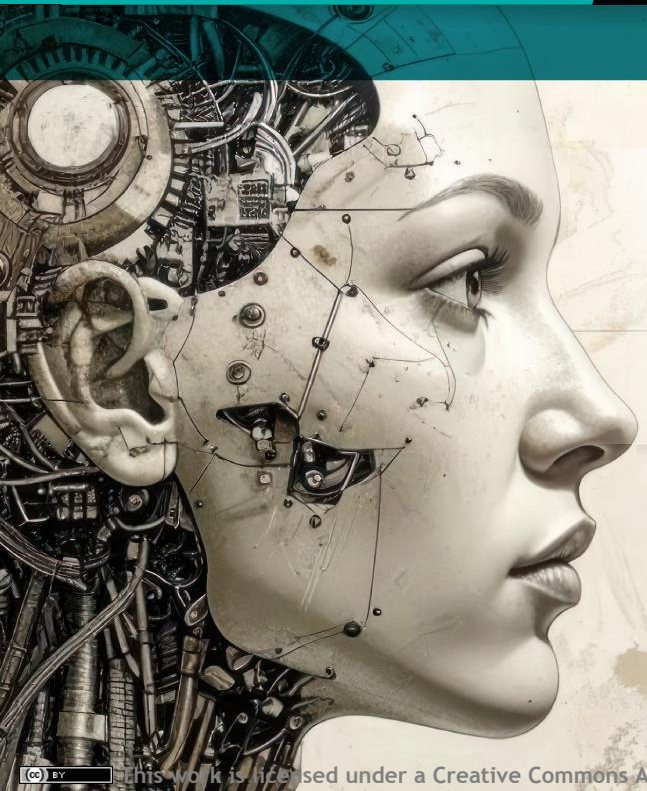
GOSIM Shenzhen 2026

KHRONOS GROUP

KHRONOS® MACHINE LEARNING



From Khronos AI Ecosystem Research Insights to Edge AI Inference with Vulkan

October 2026

Markus Tavenrath

ML Council Chair, Khronos

This work is licensed under a Creative Commons Attribution 4.0 International License

Introduction

- ML and (real-time) graphics are converging
- Every developer shipping ML in graphics is asking:
  - Can it run on my users' hardware?
  - Is it fast enough?
  - Can I keep it maintainable?



This work is licensed under a Creative Commons Attribution 4.0 International License

Khronos ML Ecosystem Research

Independent research by Real World Insights, sponsored by Futurewei Technologies

What did we do?

Stakeholder coverage

Discovery interviews and follow-ups across model developers, hardware architects, compiler engineers, and standards roles across the entire ML stack.

Why did we do it?

Understand real-world ML acceleration pain points for hardware and software vendors. Identify how standards bodies such as Khronos can reduce ML ecosystem complexity.

What did we learn?

~85% deployment effort

Interview estimate: backend acceleration dominates deployment effort.

3 portability patterns

One-to-one integrations.
One-to-many device fleets.
Many-to-one model support.

Inference

Strong fit for standards: hardware diversity, speed, and deployment reuse.

20–30% custom ops

Custom operators evolve faster than fixed operator sets can accommodate.

Neither alone

Graphs struggle with custom ops. Kernels give control but need experts.

≥80% peak target

Practitioner target: near peak performance on a given model and system.

How can we proceed?

Unified HAL

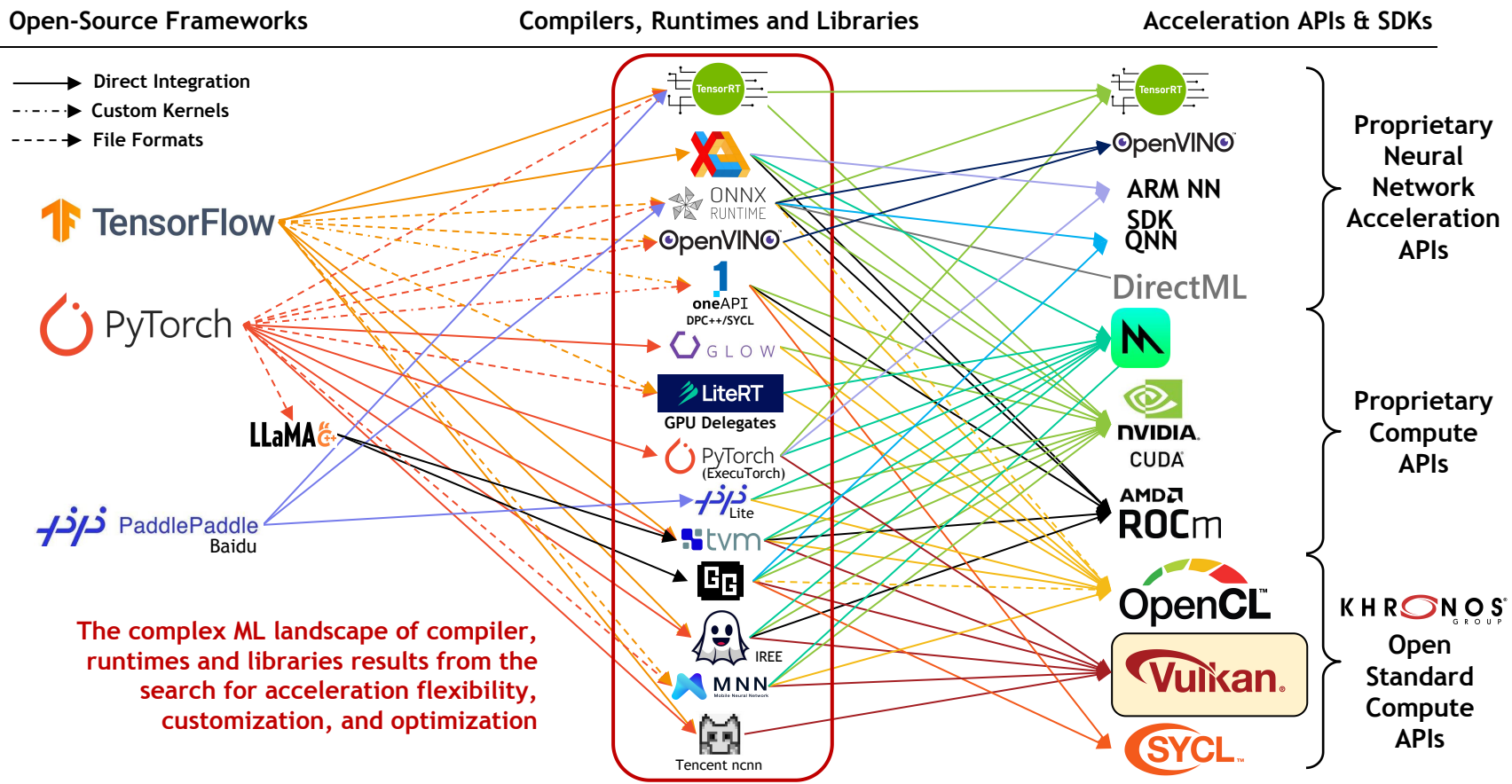
Proposed: optimized vendor paths, with an open compute fallback.

M×N → M+N

A shared interface for additive integration across frameworks and accelerators.

This work is licensed under a Creative Commons Attribution 4.0 International License

Machine Learning Acceleration Complexity



This work is licensed under a Creative Commons Attribution 4.0 International License

Application Types



Performance Sensitive
(Game Engine)

- In games every ms counts, 120hz → 8.3ms
- Every shader must run close to SOL
  - no frame budget left for inefficient AI
- Few ML networks per application
  - Hand-tuning is acceptable



Feature Sensitive
(Enterprise Application + Game Engine Editors)

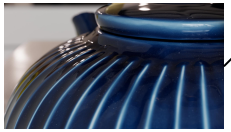
- Time to market is everything
- Deploy at >80% of SOL, sometimes lower
  - Limited budget to spare on optimization
- Every AI feature uses a different network
  - Hand-tuning is not an option

This work is licensed under a Creative Commons Attribution 4.0 International License

ML runs efficiently on Vulkan








Compute & Memory Bandwidth Intensive Operations

matmul · softmax · dequantization · normalization · and more

Runs efficiently on

Vulkan ML TSG

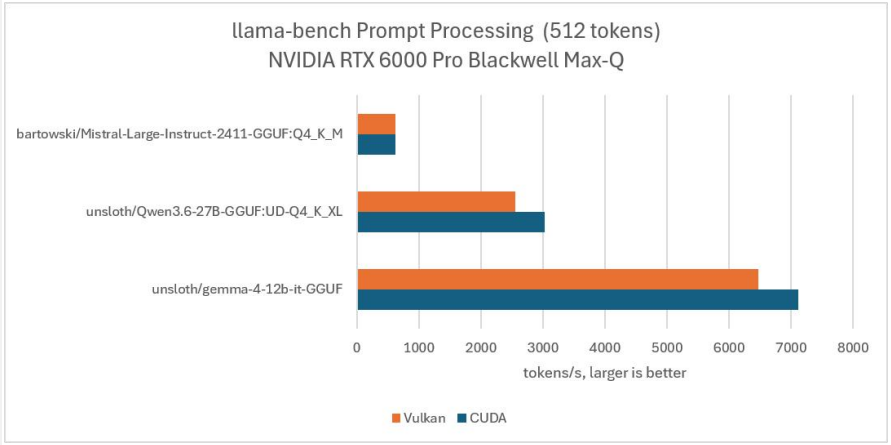
Identify Need
Develop Extensions

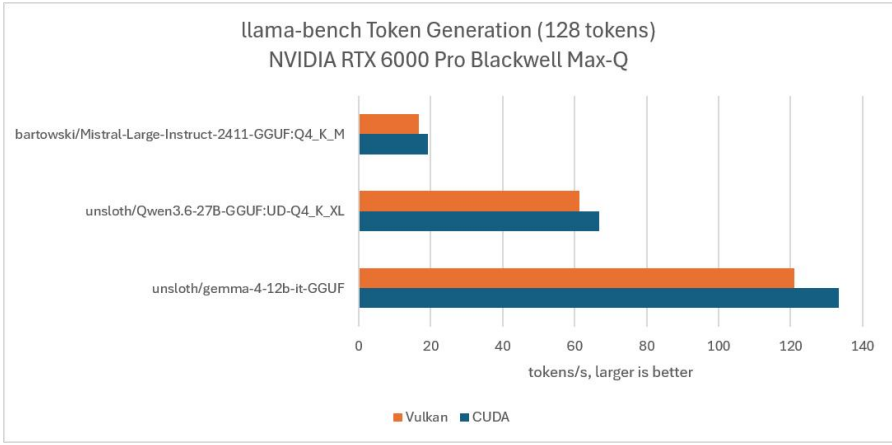
Vulkan

This work is licensed under a Creative Commons Attribution 4.0 International License

Inference Framework Performance

- **Industry expectation: 80% of native backend performance is sufficient**
  - Assuming native backend is SOL
  - Vulkan achieves 85% - 100+% of CUDA performance on most models on NVIDIA hardware
  - Performance varies by model architecture
  - Vulkan backend optimization is ongoing
- **Large models (123B) run on Vulkan**
- **Experience may differ across IHV hardware**
  - Demonstrates Vulkan's compute capabilities can enable efficient inference



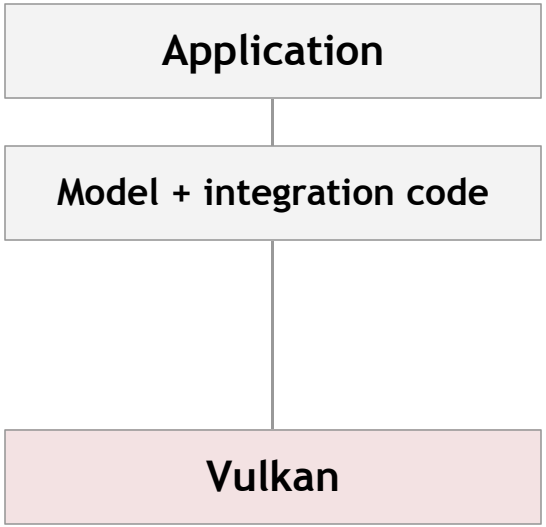


This work is licensed under a Creative Commons Attribution 4.0 International License

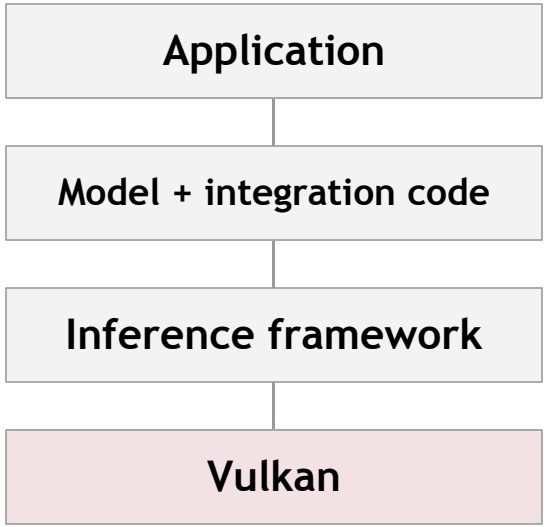
Three ways to deploy AI with Vulkan

Choose how much of the inference implementation your application owns

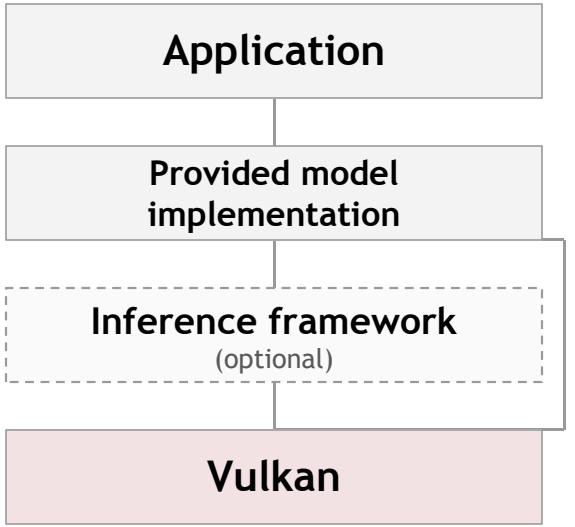
A. Direct shader integration



B. Inference framework



C. Use an existing model implementation



This work is licensed under a Creative Commons Attribution 4.0 International License

Vulkan ML Compute: Cooperative Matrices

Matrix multiplication and related operations across cooperating threads

2019	2023	2024	2026
VK_NV_ cooperative_matrix	VK_KHR_ cooperative_matrix	VK_NV_ cooperative_matrix2	VK_EXT_cooperative_ matrix_maintenance1
D = A * B + C	**D = A * B + C**	Workgroup scope Flexible matrix sizes Tensor loads + decoding Per-element operations Reductions + conversions	**Per-element operations** **Reductions** **Type / use conversions** **Element coordinates**
Subgroup matrix multiply-add	Shared interface across vendors		

2026 VK_NV_cooperative_matrix_decode_vector

Decode several quantized weights per callback during matrix loads

This work is licensed under a Creative Commons Attribution 4.0 International License

Vulkan ML Compute: Other Operations

Integer arithmetic and independent network evaluation per invocation

2021

Integer dot products

VK_KHR_shader_integer_dot_product

dot(a, b) = sum(a[i] * b[i])

Packed integer arithmetic
A building block for quantized ML kernels

2024

Cooperative vectors

VK_NV_cooperative_vector

y = W * x + b

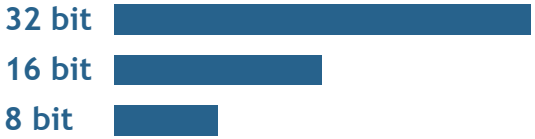
Matrix-vector operations per invocation
Independent small-network evaluation
e.g. neural texture compression

This work is licensed under a Creative Commons Attribution 4.0 International License

Vulkan Extensions for ML data types

Compact storage, shader arithmetic and lower precision ML formats

2017-2018



Smaller storage

16-bit and 8-bit buffers reduce data footprint

2017 VK_KHR_16bit_storage

2018 VK_KHR_8bit_storage

2019-2024

FP16 × FP16

BF16: 8 exponent bits

Arithmetic and range

FP16 / INT8 shader arithmetic
BF16 adds wider numerical range

2019 VK_KHR_shader_float16_int8

2024 VK_KHR_shader_bfloat16

2025-2026

FP8 / FP6 / FP4

E8M0 scale + MXINT8

Lower precision formats

FP8 plus the building blocks for OCP microscaling formats

2025 VK_EXT_shader_float8
E4M3 / E5M2

2026 VK_EXT_shader_ocp_microscaling_types

OCP adds types and conversions. Direct microscaled tensor operations require separate support.

This work is licensed under a Creative Commons Attribution 4.0 International License

Vulkan ML Data Graphs

Full ML Operator Graph Pipelines

Graph execution

One pipeline encapsulates an ML graph with tensor operations like GEMM or CONV

The application manages the session. It supplies memory and uses Vulkan descriptor sets and synchronization.

Graph foundations

VK_ARM_data_graph

Graph pipelines with SPV_ARM_graph

VK_ARM_tensors

Tensor resources alongside buffers and images

Additional graph capabilities

TOSA 1.0 operators in SPIR-V

VK_ARM_data_graph_instruction_set_tosa

Optical flow

VK_ARM_data_graph_optical_flow

QNN models and built-in models

VK_QCOM_data_graph_model

Arm neural accelerator statistics

VK_ARM_data_graph_neural_accelerator_statistics

This work is licensed under a Creative Commons Attribution 4.0 International License

A. Inference directly in shaders

You implement the network math and the code that drives it

Network evaluation

Network layers and shader operations

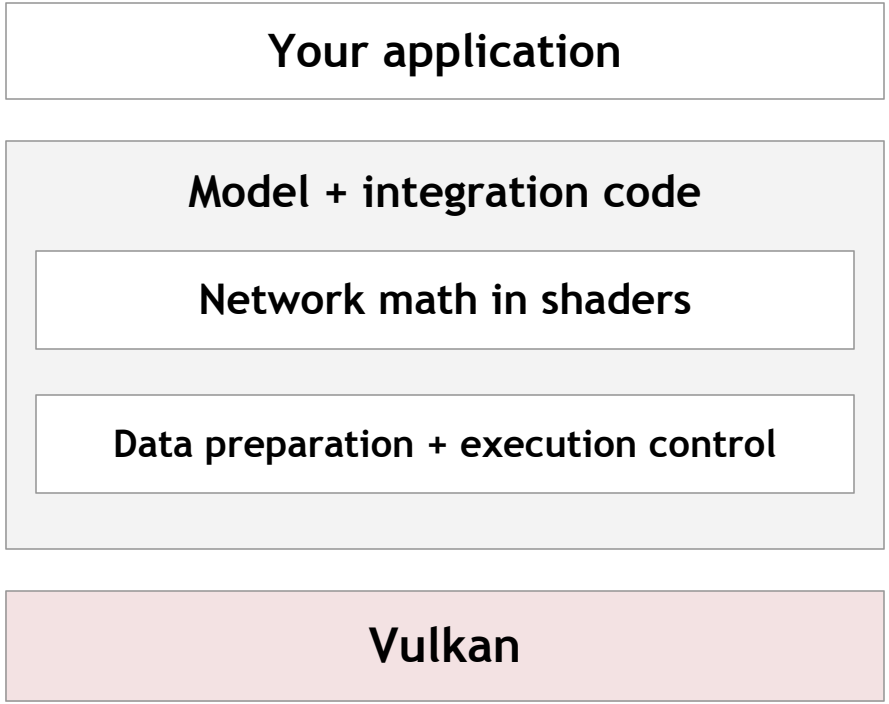
Weights, precision, and data layout

Code around the model

Weight loading and resource setup

Input and output preparation

Dispatch, synchronization, and state



This work is licensed under a Creative Commons Attribution 4.0 International License

Neural Texture Compression

- **Textures stored as latent feature maps instead of traditional texels**
  - A tiny neural network (MLP) decodes each texel at runtime using cooperative vectors
- **Higher compression ratios than BCn formats**
  - NTC at 4096×4096 requires 3.8 MB — less than BC high at 1024×1024 (5.3 MB)
  - Reference uncompressed: 4096×4096 at 256 MB
  - Better PSNR: 22.0 dB (NTC) vs 19.4 dB (BC high)
- **Supports high channel count materials, efficiently compressing multi-channel data**
- **Reduces disk footprint, download bandwidth, and VRAM usage**



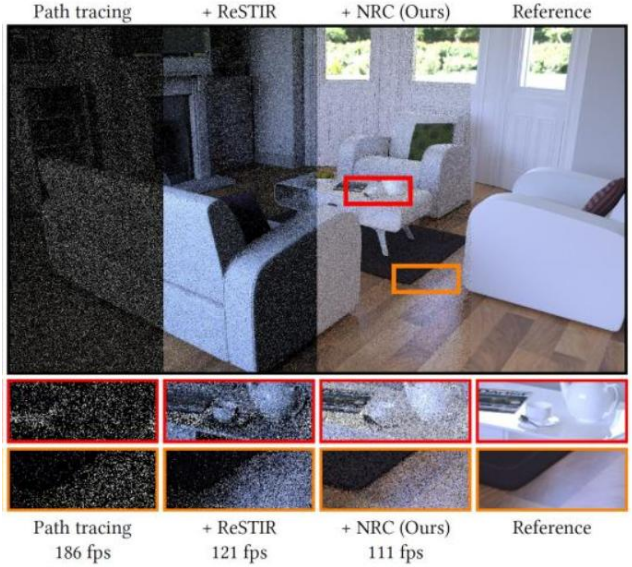
https://github.com/NVIDIA-RTX/RTXNTC



This work is licensed under a Creative Commons Attribution 4.0 International License

Real-time Neural Radiance Caching

- Replaces expensive path tracing bounces with a small neural network query
- Network learns the scattered radiance function
  - position, normal, direction, material → incoming light
- Trains itself online every frame using extended path samples as ground truth
- Converges in seconds
  - no offline training phase
- Implemented in open GLSL using VK_KHR_cooperative_matrix
- Vulkan Implementation can be found here:
  - https://github.com/AdamYuan/VkNRC



https://research.nvidia.com/publication/2021-06_real-time-neural-radiance-caching-path-tracing

This work is licensed under a Creative Commons Attribution 4.0 International License

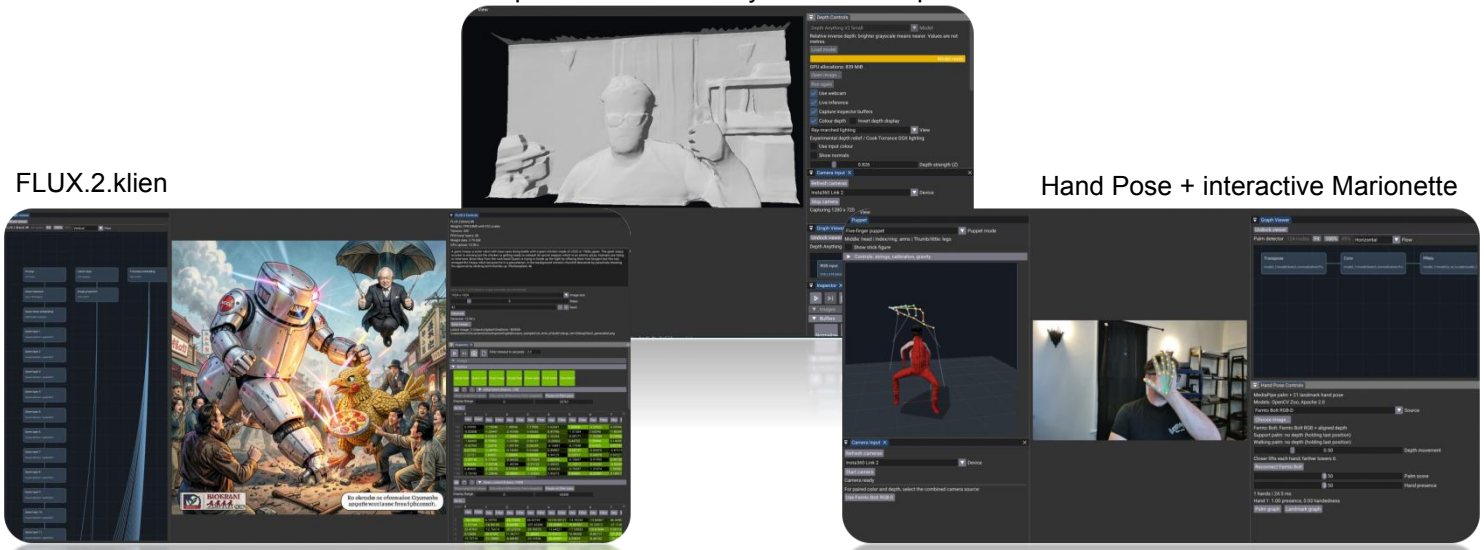
VK-MINI AI

- Eight inference samples built with Vulkan, Slang and nvpro_core2.
- Workloads span image generation, local language models, object detection, depth, segmentation and hand pose.
- Each sample pairs a real model with an interactive graphics application.

Depth Estimation + Ray marched output.

FLUX.2.klien

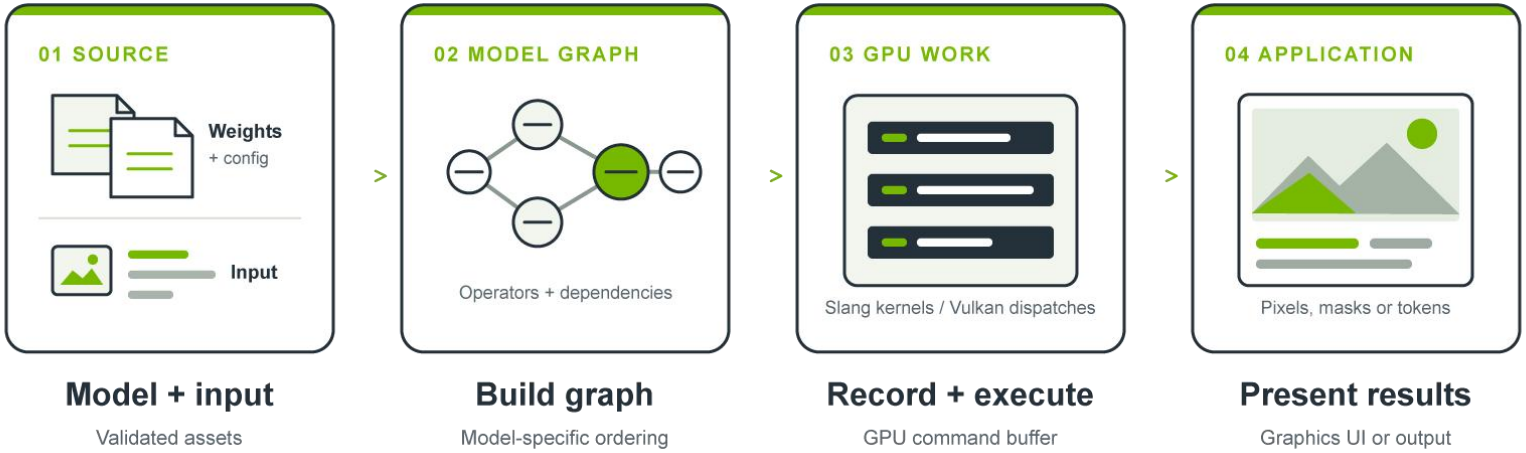
Hand Pose + interactive Marionette



This work is licensed under a Creative Commons Attribution 4.0 International License

From model weights to GPU work

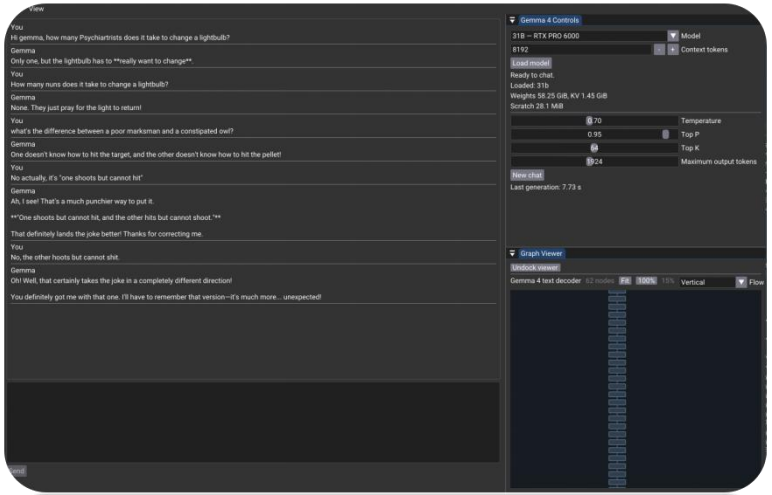
- Load validated model weights and build an ordered execution graph.
- Record Slang compute operators into Vulkan command buffers.
- Reuse tensor and GPU utilities while keeping model-specific kernels where they matter.



This work is licensed under a Creative Commons Attribution 4.0 International License

Where inference meets rendering

- Depth estimates drive an interactive, relit 2.5D relief.
- Hand landmarks can drive a rigged character; detection and segmentation add scene understanding.
- The samples provide starting points for applications that combine neural inference with graphics.
- Non rendering samples include
  - Gemma4 LLM
  - Sample Agentic system incorporating
    - Gemma4,
    - RAG,
    - MCP server
    - external MCP integrations



- Release planned : 28th October 2026

This work is licensed under a Creative Commons Attribution 4.0 International License

B. Deploy models through a framework

You supply the model; the framework handles its supported inference operations

Your application provides

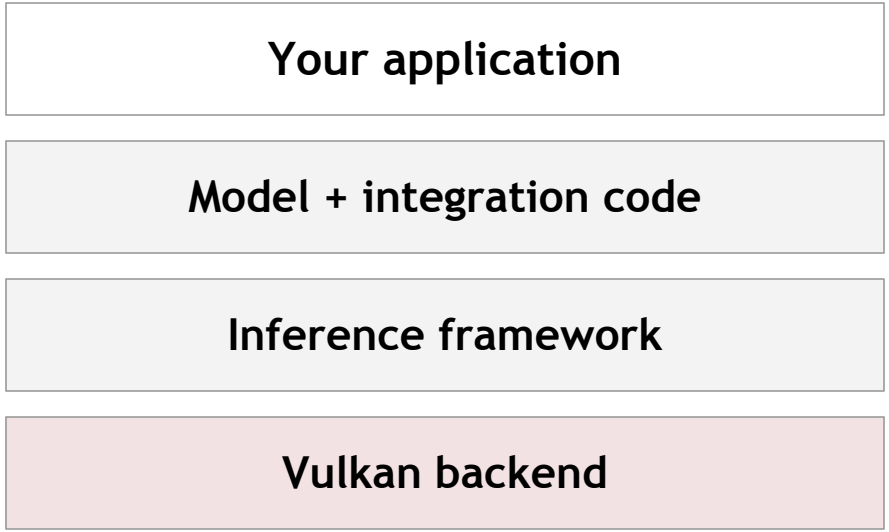
A model in a supported format

Input and output handling

Integration with the inference API

The framework provides

Graph execution, operator kernels, and backend dispatch



KHRONOS GROUP

This work is licensed under a Creative Commons Attribution 4.0 International License

B. Inference framework options

Inference frameworks with Vulkan backends



ncnn



ggml





ExecuTorch

KHRONOS GROUP

This work is licensed under a Creative Commons Attribution 4.0 International License

Superresolution

- **Real-ESRGAN**
  - Enhanced Super-Resolution Generative Adversarial Network
- **Upscales images 2× or 4× — reconstructs realistic high-frequency detail via GAN training**
- **Robust to real-world degradations: blur, noise, compression artifacts**
- **Convolution-heavy RRDB architecture maps directly to Vulkan cooperative matrix**
- **Runs via ncnn Vulkan backend — no proprietary runtime required**

Executable only: ≈6.0 MB

Real-ESRGAN, Windows x64, Vulkan. Model weights excluded.



4x upscaler



https://github.com/xinntao/Real-ESRGAN-ncnn-vulkan

This work is licensed under a Creative Commons Attribution 4.0 International License

Video Frame Interpolation

- **Real-Time Intermediate Flow Estimation for Video Frame Interpolation**
  - Temporal interpolation generates a frame between two existing frames
- **Heavy convolutional networks**
  - IFNet to directly estimate optical flow
  - FusionNet to blend the warped frames
  - Well suited to cooperative-matrix execution

Executable only: ≈6.8 MB (Windows x64, Vulkan)
Original - 17 frames



4x frame interpolation
rife-ncnn-vulkan -m rife-v4 -i input -o output -s 0.25 -n 69



https://github.com/nihui/rife-ncnn-vulkan

© copyright 2008, Blender Foundation / www.bigbuckbunny.org — CC Attribution 3.0

This work is licensed under a Creative Commons Attribution 4.0 International License

C. Existing model implementations

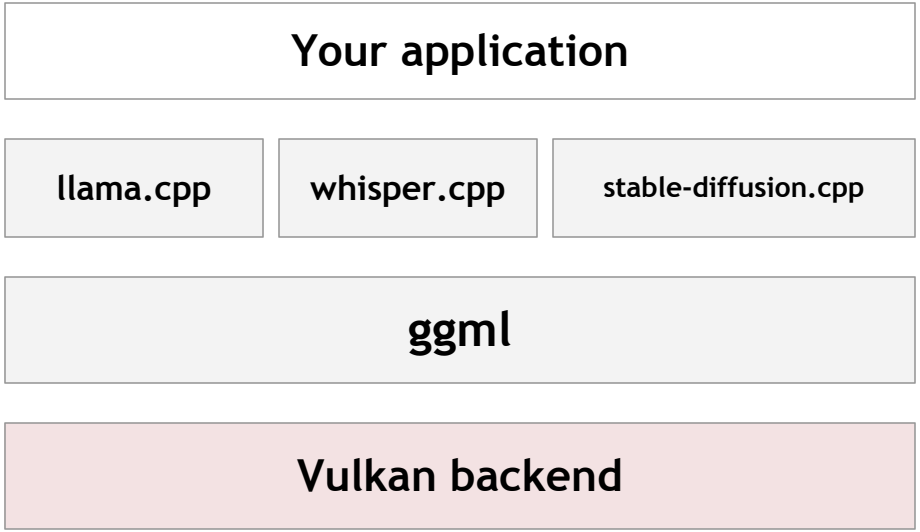
The ggml ecosystem provides model-specific inference code across many workloads

Reuse the model implementation

Choose a supported model family.

Integrate the project's API or tool.

Provide compatible model weights.



A shared optimization point

Backend improvements can benefit projects as they adopt ggml updates.

Build size (Windows x64, b5d8120)

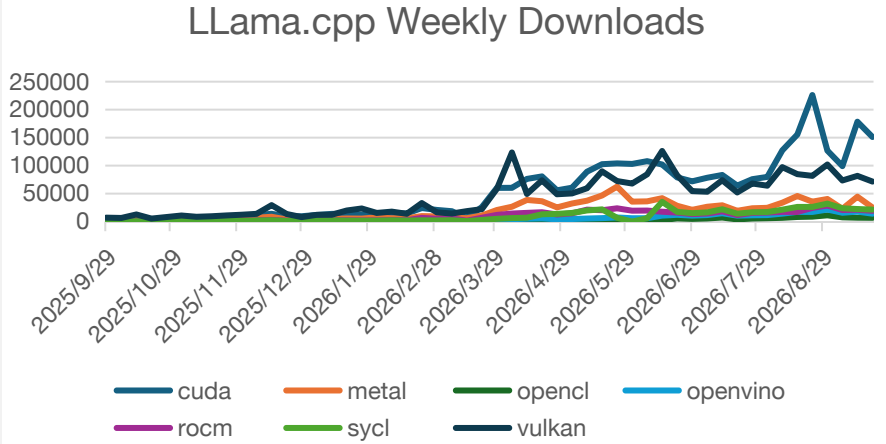
Multi-backend build: ≈2.0 GB
Vulkan-only build: 46.81 MB

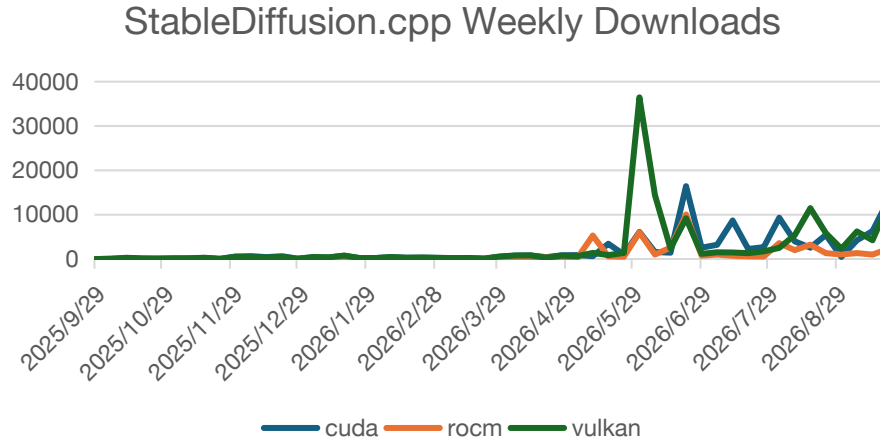
Binary size only. Model weights excluded.

This work is licensed under a Creative Commons Attribution 4.0 International License

GGML market observations

- **llama.cpp downloads jumped around April**
  - Vulkan ranks second in weekly downloads
- **stable-diffusion.cpp downloads jumped around June**
  - CUDA and Vulkan have equal download share
- **Vulkan is now the default GPU backend for llama.cpp**
  - One Vulkan backend handles vendor capabilities internally





This work is licensed under a Creative Commons Attribution 4.0 International License

GGML ECOSYSTEM

LANGUAGE + PRIVACY

llama.cpp

privacy-filter.cpp

SPEECH RECOGNITION

whisper.cpp

parakeet.cpp

moss-transcribe.cpp

SPEECH GENERATION

qwentts.cpp

vibevoice.cpp

moss-tts.cpp

magpie-tts.cpp

IMAGE + VIDEO

stable-diffusion.cpp

talking-portrait.cpp

AUDIO ANALYSIS

LocalVQE

ced.cpp

voice-detect.cpp

COMPUTER VISION

depth-anything.cpp

locate-anything.cpp

rf-detr.cpp

face-detect.cpp

3D GEOMETRY

trellis.cpp

free-splatter.cpp

sam3d.cpp

MOTION + CHARACTERS

gem-x.cpp

kimodo.cpp

motion-bricks.cpp

skin-tokens.cpp

animate-any-mesh.cpp

This work is licensed under a Creative Commons Attribution 4.0 International License

Image and Video Generation and Manipulation

- **stable-diffusion.cpp — the llama.cpp for images and video**
  - Same philosophy, same engine (ggml) — but for images and video
- **Generate images from text prompts (txt2img)**
- **Edit images using text prompts (img2img)**
  - Inpainting: replace a masked region within an image
  - Outpainting: extend an image beyond its original borders
- **Guided generation with ControlNet**
- **Upscale generated images with ESRGAN**
- **Generate videos (Wan, LTX models)**

It's rapidly evolving!
30+ Image&Image Edit Models
6 Video Models

2026/09 Qwen-Image-2.1 (Day-0 Support)
2026/08 LTX-2.5
2026/08 MiniMax-H3 (Day-1 Support)
2026/06 Krea2
2026/06 Ideogram4
2026/05 PiD
2026/05 Lens
2026/05 LTX-2.3
2026/01 FLUX.2-klein
2025/12 Z-Image
2025/11 FLUX.2-dev
2025/10 Qwen-Image-Edit
2025/10 Qwen-Image
2025/09 Wan2.1 Vace
2025/09 Wan2.1 / Wan2.2



Prompt to img2img:

replace the dog by a grumpy cat, keep the characters in the background



https://github.com/leejet/stable-diffusion.cpp

This work is licensed under a Creative Commons Attribution 4.0 International License

One Vulkan path across the ecosystem

The research asked for a unified path. Vulkan provides one today.

One backend, one host API

The backend handles device capabilities internally,
using different shader paths when required.

Portable deployment across vendors

One Vulkan build can cover supported GPUs across vendors
without bundling separate vendor compute runtimes.

Increasing support across the ecosystem

Direct shaders, inference frameworks, and model implementations
already use this shared path.

This work is licensed under a Creative Commons Attribution 4.0 International License

GOSIM

GOSIM Shenzhen 2026

KHRONOS®
MACHINE
LEARNING

KHRONOS®
GROUP

For ML questions and feedback
ml@khronos.org

More Information on all things Khronos!
www.khronos.org
memberservices@khronosgroup.org



GOSIM

GOSIM Shenzhen 2026

KHRONOS®
MACHINE
LEARNING

KHRONOS®
GROUP

For ML questions and feedback
ml@khronos.org

More Information on all things Khronos!
www.khronos.org
memberservices@khronosgroup.org



GOSIM

GOSIM Shenzhen 2026

KHRONOS®
MACHINE
LEARNING

KHRONOS®
GROUP

For ML questions and feedback
ml@khronos.org

More Information on all things Khronos!
www.khronos.org
memberservices@khronosgroup.org

